

KI-Einsatz & Prompt-Dokumentation: IronLog

Projekt: IronLog – Minimalist Gym Tracker

KI-Modell: Google Gemini (Modellversion 3.1 Pro)

Rolle der KI: Coding Partner

1. Strategie des KI-Einsatzes

Die KI wurde nicht zum blinden Generieren von Code genutzt, sondern als interaktiver "Pair Programmer". Der Fokus lag auf:

- **Architektur-Planung:** Validierung des Datenbank-Schemas und Aufbau einer sauberen MVC/Service-Struktur.
- **Seeding & Models:** Generierung großer Datensätze (70+ Übungen) sowie das Schreiben von fehlerfreien Dart-Datenklassen (Models) für das Frontend.

2. Exemplarische Prompts & Ergebnisse

Hier sind die entscheidenden Prompts, die zur finalen Backend-Struktur und zum Frontend-Fundament geführt haben.

Phase 1: Datenbank-Architektur & Basis

- **Ziel:** Erstellung eines robusten SQLite-Schemas, das über einfache CRUD-Operationen hinausgeht.
- **Prompt:** *"Erstelle ein Express.js Backend mit SQLite. Ich brauche Tabellen für Übungen und Workouts. Füge außerdem Tabellen für Trainingspläne (Routinen) und Körpermaße hinzu."*
- **Ergebnis:** Ein relationales Modell mit 6 Tabellen (exercises, workouts, routines, routine_exercises, measurements, strength_standards), korrekt verknüpft über Foreign Keys.

Phase 2: Data Seeding (Automatisierung)

- **Ziel:** Befüllung der Datenbank mit realistischen Inhalten, um manuelle Eingabe zu vermeiden.
- **Prompt:** *"Fülle die Datenbank mit den Top 75 Fitness-Übungen. Bitte sortiere sie nach Muskelgruppen (Brust, Rücken, Beine, etc.) und füge jeder Übung eine sinnvolle Pausenzeit hinzu (z. B. 180s für schwere Verbundübungen, 60s für Isolation)."*
- **Ergebnis:** Ein SQL-Transaktions-Skript, das automatisch 75 Übungen mit Metadaten (default_rest_seconds) in die Datenbank lädt.

Phase 3: Gamification & Logik

- **Ziel:** Implementierung eines Features, das den Nutzer motiviert (Vergleich mit Weltstandards).
- **Prompt:** *"Ich möchte ein Ranking-System. Wenn der User beim Bankdrücken 100kg schafft, soll das Backend berechnen, zu welchen Top % er gehört (z.B. Advanced oder Elite). Erstelle eine Logik, die den User-Rekord mit einer Referenztabelle vergleicht und ein Ranking inkl. Icon zurückgibt."*
- **Ergebnis:** Neue Tabelle strength_standards. API-Endpunkt GET /api/stats/ranking/:id. Komplexe SQL-Abfrage, die das Maximalgewicht (MAX(weight_kg)) ermittelt und dynamisch in Kategorien (Beginner bis Elite) einordnet.

Phase 4: Backend-Refactoring (Saubere Architektur)

- **Ziel:** Auflösung der index.js in eine übersichtlichere Ordnerstruktur.
- **Prompt:** *"Wie mache ich die Struktur meines backends also dass es schöner ist. Mach es mit einem src Ordner und den unterordnern: middleware, models, services."*
- **Ergebnis:** Aufteilung des Codes in database.js und spezifische Service-Routen (exerciseService.js, workoutService.js, etc.), die in der index.js importiert werden.

Phase 5: Frontend-Architektur & API-Anbindung (Flutter)

- **Ziel:** Aufbau eines strukturierten Flutter-Projekts und Verknüpfung mit dem lokalen Node.js-Backend.
- **Prompt:** *"Wir machen jetzt schritt für schritt die flutter app zu diesem backend: sie soll gut strukturiert sein und alle Features des backends verwenden."*
- **Ergebnis:** Definition der lib/-Struktur (core, models, services, screens), Einbindung von http und fl_chart, sowie die automatische Generierung exakt passender Dart-Datenklassen (Exercise, Workout, etc.) für das JSON-Parsing.

Phase 6: UI-Redesign (Hevy-Stil & Logger-Logik)

- **Ziel:** Umstellung des Workout-Loggers auf ein listenbasiertes System mit Fokus auf Usability.
- **Prompt:** „Mache es anders: Bei 'Workouts' orientiere dich an der App 'Heavy'. Es soll so ungefähr sein wie bei 'Heavy', mit einem Timer, der erst startet, wenn eine Übung hinzugefügt wurde, und einem Button 'Workout speichern'.“
- **Ergebnis:** Ein dynamischer WorkoutScreen, der Übungstracker-Objekte verwaltet. Er enthält eine integrierte Suchfunktion im Bottom-Sheet, Checkbox-Logik für abgeschlossene Sätze und eine zentralisierte Timer-Steuerung.

Phase 7: Erweiterte Übungsdetails & Backend-Anpassung

- **Ziel:** Integration von Fachwissen (Anleitungen) direkt in die App-Oberfläche.
- **Prompt:** „Mache, dass wenn ich auf eine Übung klicke, eine kleine Beschreibung bekomme, wie sie geht, auf was man achten muss und ein Bild der Ausführung. Sag mir, wie ich mein Backend dafür updaten muss.“

- **Ergebnis:** Erweiterung des SQLite-Schemas in database.js um die Felder description, tips und image_url. Implementierung einer Detail-Ansicht im LibraryScreen, die diese Daten über ein interaktives Bottom-Sheet visualisiert.

Phase 8: Datenvisualisierung & Analyse (Progress)

- **Ziel:** Umfassende grafische Aufarbeitung der Trainingserfolge.
- **Prompt:** „Mache nun den Progress Screen. Orientiere dich wieder an 'Heavy' und benutze alle Features meines Backends, auch wenn es mehr Funktionen bietet als die Vorlage.“
- **Ergebnis:** Ein Statistik-Dashboard unter Nutzung von fl_chart. Es verknüpft die Backend-Endpunkte für den Gewichtsverlauf (Line-Chart), die Muskelverteilung (Pie-Chart), Personal Records und das globale Ranking-System in einer übersichtlichen Ansicht.

Phase 9: App-Einstellungen & Finalisierung

- **Ziel:** Vervollständigung der Kern-Navigation und Nutzer-Präferenzen.
- **Prompt:** „Erstelle mir nun den Settings Screen mit Profil, Dunkelmodus-Option und Einheiten-Wechsel (kg/lbs).“
- **Ergebnis:** Erstellung des SettingsScreen, der das visuelle Design der App abrundet und Platzhalter für die Account-Verwaltung sowie die Steuerung der App-Einheiten bietet.

Phase 10: Lokalisierung & UX-Feinschliff

- **Ziel:** Optimierung der Nutzererfahrung für den deutschsprachigen Raum.
- **Prompt:** „Stelle den Text vom ganzen Screen auf Deutsch um und füge im 'Select Exercise' Fenster eine Suchleiste hinzu.“
- **Ergebnis:** Vollständige Lokalisierung aller UI-Komponenten und Implementierung einer Echtzeit-Filterung (StatefulBuilder) innerhalb der Übungsauswahl zur schnelleren Navigation bei großen Datensätzen.